A Practical Guide to Sequence to sequence learning

TOPIC -- SEQUENCE TO SEQUENCE LEARNING

-- 01 --

(h 0 , h 1 , … {\displaystyle h_{0},h_{1},\dots } , "<start>") → "la" (h 0 , h 1 , … {\displaystyle h_{0},h_{1},\dots } , "<start> la") → "la zone" (h 0 , h 1 , … {\displaystyle h_{0},h_{1},\dots } , "<start> la zone") → "la zone de" ... (h 0 , h 1 , … {\displaystyle h_{0},h_{1},\dots } , "<start> la zone de contrôle international") → "la zone de contrôle international <end>" Here, we use the special <start> token as a control character to delimit the start of input for the decoder. The decoding terminates as soon as "<end>" appears in the decoder output.

-- 02 --

This is the dot-attention mechanism. The particular version described in this section is "decoder cross-attention", as the output context vector is used by the decoder, and the input keys and values come from the encoder, but the query comes from the decoder, thus "cross-attention".

-- 03 --

Priority dispute One of the papers cited as the originator for seq2seq is (Sutskever et al 2014), published at Google Brain while they were on Google's machine translation project. The research allowed Google to overhaul Google Translate into Google Neural Machine Translation in 2016. Tomáš Mikolov claims to have developed the idea (before joining Google Brain) of using a "neural language model on pairs of sentences... and then [generating] translation after seeing the first sentence"—which he equates with seq2seq machine translation, and to have mentioned the idea to Ilya Sutskever and Quoc Le (while at Google Brain), who failed to acknowledge him in their paper. Mikolov had worked on RNNLM (using RNN for language modelling) for his PhD thesis, and is more notable for developing word2vec.

-- 04 --

seq2seq is an approach to machine translation (or more generally, sequence transduction) with roots in information theory, where communication is understood as an encode-transmit-decode process, and machine translation can be studied as a special case of communication. This viewpoint was elaborated, for example, in the noisy channel model of machine translation. In practice, seq2seq maps an input sequence into a real-numerical vector by using a neural

network (the encoder), and then maps it back to an output sequence using another neural network (the decoder). The idea of encoder-decoder sequence transduction had been developed in the early 2010s. The papers most commonly cited as the originators that produced seq2seq are two papers from 2014. In the seq2seq as proposed by them, both the encoder and the decoder were LSTMs. This had the "bottleneck" problem, since the encoding vector has a fixed size, so for long input sequences, information would tend to be lost, as they are difficult to fit into the fixed-length encoding vector. The attention mechanism, proposed in 2014, resolved the bottleneck problem. They called their model RNNsearch, as it "emulates searching through a source sentence during decoding a translation". A problem with seq2seq models at this point was that recurrent neural networks are difficult to parallelize. The 2017 publication of Transformers resolved the problem by replacing the encoding RNN with self-attention Transformer blocks ("encoder blocks"), and the decoding RNN with cross-attention causally-masked Transformer blocks ("decoder blocks").

-- 05 --

Seq2seq is a family of machine learning approaches used for natural language processing. Originally developed by Lê Vi■t Qu■c, a Vietnamese computer scientist and a machine learning pioneer at Google Brain, this framework has become foundational in many modern AI systems. Applications include language translation, image captioning, conversational models, speech recognition, and text summarization. Seq2seq uses sequence transformation: it turns one sequence into another sequence.

-- 06 --

The encoder is responsible for processing the input sequence and capturing its essential information, which is stored as the hidden state of the network and, in a model with attention mechanism, a context vector. The context vector is the weighted sum of the input hidden states and is generated for every time instance in the output sequences.

-- 07 --

The decoder takes the context vector and hidden states from the encoder and generates the final output sequence. The decoder operates in an autoregressive manner, producing one element of the output sequence at a time. At each step, it considers the previously generated elements, the context vector, and the input sequence information to make predictions for the next element in the output sequence. Specifically, in a model with attention mechanism, the context vector and the hidden state are concatenated together to form an attention hidden

vector, which is used as an input for the decoder. The seq2seq method developed in the early 2010s uses two neural networks: an encoder network converts an input sentence into numerical vectors, and a decoder network converts those vectors to sentences in the target language. The Attention mechanism was grafted onto this structure in 2014 and is shown below. Later it was refined into the encoder-decoder Transformer architecture of 2017.

-- 08 --

More succinctly, we can write it as c 0 = A t t e n t i o n (h 0 d W Q , H W K , H W V) = s o f t m a x ((h 0 d W Q) (H W K) T) (H W V) {\displaystyle c_{0}=\mathrm {Attention} (h_{0}^{d}W^{Q},HW^{K},HW^{V})=\mathrm {softmax} ((h_{0}^{d}W^{Q})\;(HW^{K})^{T})(HW^{V})} where the matrix H {\displaystyle H} is the matrix whose rows are h 0 , h 1 , … {\displaystyle h_{0},h_{1},\dots } . Note that the querying vector, h 0 d {\displaystyle h_{0}^{d}} , is not necessarily the same as the key-value vector h 0 {\displaystyle h_{0}} . In fact, it is theoretically possible for query, key, and value vectors to all be different, though that is rarely done in practice.

-- 09 --

There is a subtle difference between training and prediction. During training time, both the input and the output sequences are known. During prediction time, only the input sequence is known, and the output sequence must be decoded by the network itself. Specifically, consider an input sequence x 1 : n {\displaystyle x_{1:n}} and output sequence y 1 : m {\displaystyle y_{1:m}} . The encoder would process the input x 1 : n {\displaystyle x_{1:n}} step by step. After that, the decoder would take the output from the encoder, as well as the <bos> as input, and produce a prediction y ^ 1 {\displaystyle {\hat {y}}_{1}} . Now, the question is: what should be input to the decoder in the next step? A standard method for training is "teacher forcing". In teacher forcing, no matter what is output by the decoder, the next input to the decoder is always the reference. That is, even if y ^ 1 ≠ y 1 {\displaystyle {\hat {y}}_{1}\neq y_{1}} , the next input to the decoder is still y 1 {\displaystyle y_{1}} , and so on. During prediction time, the "teacher" y 1 : m {\displaystyle y_{1:m}} would be unavailable. Therefore, the input to the decoder must be y ^ 1 {\displaystyle {\hat {y}}_{1}} , then y ^ 2 {\displaystyle {\hat {y}}_{2}} , and so on. It is found that if a model is trained purely by teacher forcing, its performance would degrade during prediction time, since generation based on the model's own output is different from generation based on the teacher's output. This is called exposure bias or a train/test distribution shift. A 2015 paper recommends that, during training, randomly switch between teacher forcing and no teacher forcing.